

```
In [34]: #231FA04B07-BATCHU VENKATESH, SEC-6
#1.Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, MinMaxScaler

%matplotlib inline
```

```
In [17]: #2.Load the Dataset
df = pd.read_csv(R"C:\Users\yaswa\Desktop\heart-4B07.csv")
```

```
In [18]: #3.A.Display the First Five Records
df.head()
```

```
Out[18]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

```
In [19]: #3.B.Display the Last Five Records
df.tail()
```

```
Out[19]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca
1020	59	1	1	140	221	0	1	164	1	0.0	2	0
1021	60	1	0	125	258	0	0	141	1	2.8	1	1
1022	47	1	0	110	275	0	0	118	1	1.0	1	1
1023	50	0	0	110	254	0	0	159	0	0.0	2	0
1024	54	1	0	120	188	0	1	113	0	1.4	1	1

```
In [20]: #4.Determine the Dimensions of the Dataset
print("Shape of Dataset:", df.shape)
```

Shape of Dataset: (1025, 14)

```
In [21]: #5. Display the names and data types of all a ributes.
df.columns
```

```
Out[21]: Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
               'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
              dtype='object')
```

In [22]: `df.dtypes`

```
Out[22]: age          int64
sex            int64
cp             int64
trestbps       int64
chol           int64
fbs            int64
restecg        int64
thalach        int64
exang          int64
oldpeak        float64
slope          int64
ca             int64
thal           int64
target         int64
dtype: object
```

In [23]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1025 non-null   int64
1   sex         1025 non-null   int64
2   cp          1025 non-null   int64
3   trestbps    1025 non-null   int64
4   chol        1025 non-null   int64
5   fbs         1025 non-null   int64
6   restecg     1025 non-null   int64
7   thalach     1025 non-null   int64
8   exang       1025 non-null   int64
9   oldpeak     1025 non-null   float64
10  slope       1025 non-null   int64
11  ca          1025 non-null   int64
12  thal        1025 non-null   int64
13  target      1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

In [24]: *#6. Iden fy numerical and categorical features.*

```
numerical_features = df.select_dtypes(include=['int64','float64'])
print(numerical_features.columns.tolist())
```

```
['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
```

```
In [12]: categorical_features = [
        'sex','cp','fbs','restecg',
        'exang','slope','ca','thal','target'
    ]
print(categorical_features)
```

```
['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal', 'target']
```

In [25]: *#7. Iden fy the target variable.*

```
target = "target"
```

```
print("Target Variable:", target)
df[target].value_counts()
```

Target Variable: target

```
Out[25]: target
1      526
0      499
Name: count, dtype: int64
```

```
In [26]: #8. Generate descriptive Statistics for all numerical attributes.
df.describe()
```

```
Out[26]:
```

	age	sex	cp	trestbps	chol	fbs
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000	0.149268
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000	0.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000	0.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000	0.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000

```
In [27]: #9. Check for missing values.
df.isnull().sum()
```

```
Out[27]: age      0
sex        0
cp         0
trestbps   0
chol       0
fbs        0
restecg    0
thalach    0
exang      0
oldpeak    0
slope      0
ca         0
thal       0
target     0
dtype: int64
```

```
In [28]: #Total missing values:
df.isnull().sum().sum()
```

```
Out[28]: 0
```

```
In [ ]: #Exploratory Data Analysis (EDA)- Aim-To perform exploratory data analysis on t
```

```
In [29]: #10. Identify and report duplicate records in the dataset.
duplicates = df.duplicated().sum()
print("Duplicate Rows:", duplicates)
```

Duplicate Rows: 723

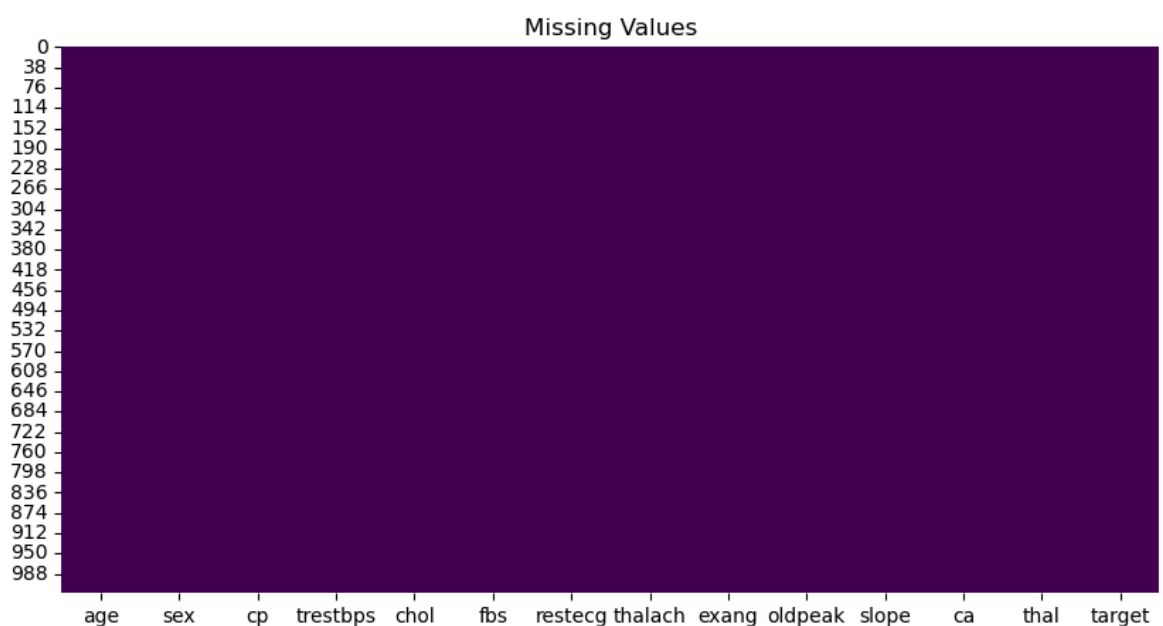
In [30]: `df[df.duplicated()]`

Out[30]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca
15	34	0	1	118	210	0	1	192	0	0.7	2	0
31	50	0	1	120	244	0	1	162	0	1.1	2	0
43	46	1	0	120	249	0	0	144	0	0.8	2	0
55	55	1	0	140	217	0	1	111	1	5.6	0	0
61	66	0	2	146	278	0	0	152	0	0.0	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	2	0
1021	60	1	0	125	258	0	0	141	1	2.8	1	1
1022	47	1	0	110	275	0	0	118	1	1.0	1	1
1023	50	0	0	110	254	0	0	159	0	0.0	2	0
1024	54	1	0	120	188	0	1	113	0	1.4	1	1

723 rows × 14 columns

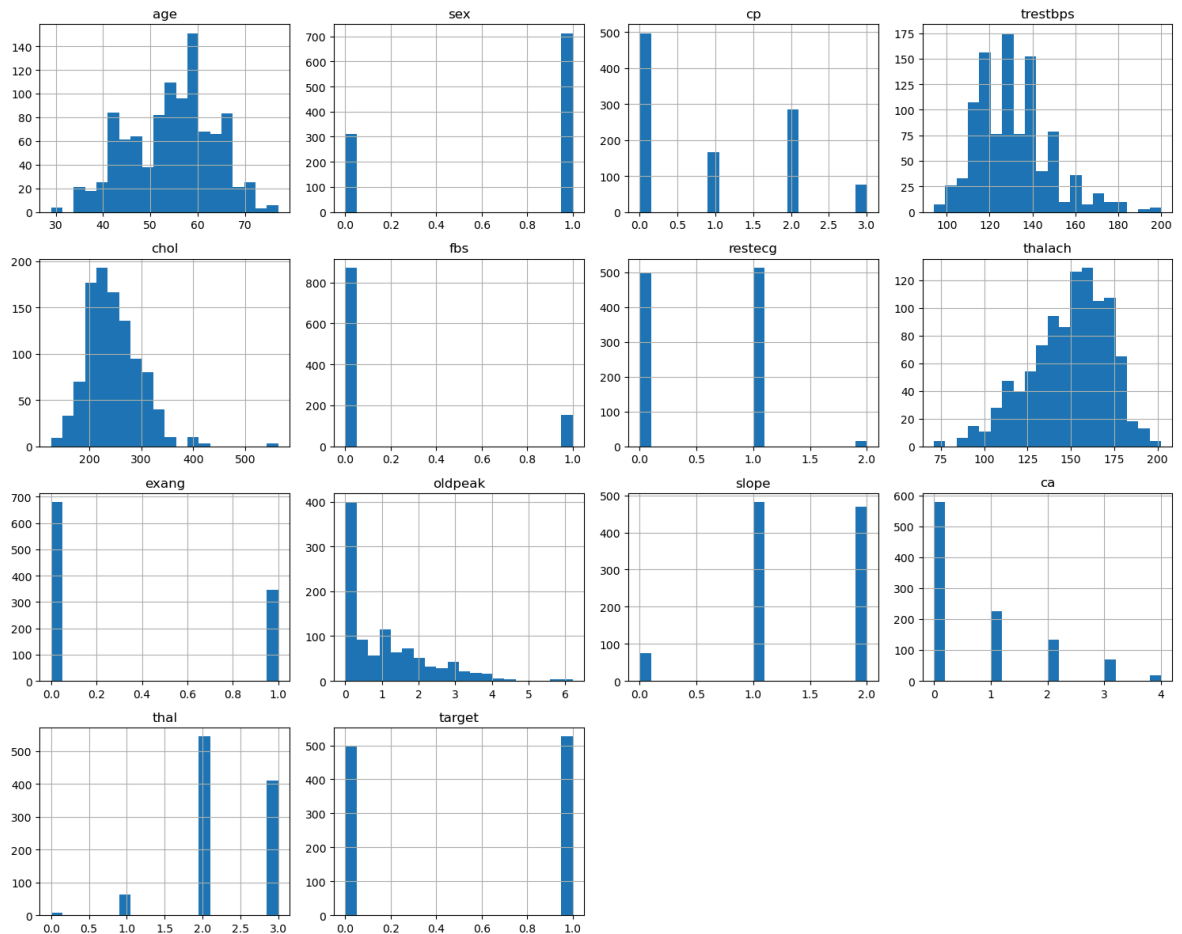
In [35]: `#11. Analyze and visualize missing values using suitable visualization techniques`
`plt.figure(figsize=(10,5))`
`sns.heatmap(df.isnull(),`
`cmap="viridis",`
`cbar=False)`
`plt.title("Missing Values")`
`plt.show()`



In [21]: `#12. Perform univariate analysis for all numerical attributes.`
`df.hist(figsize=(15,12), bins=20)`

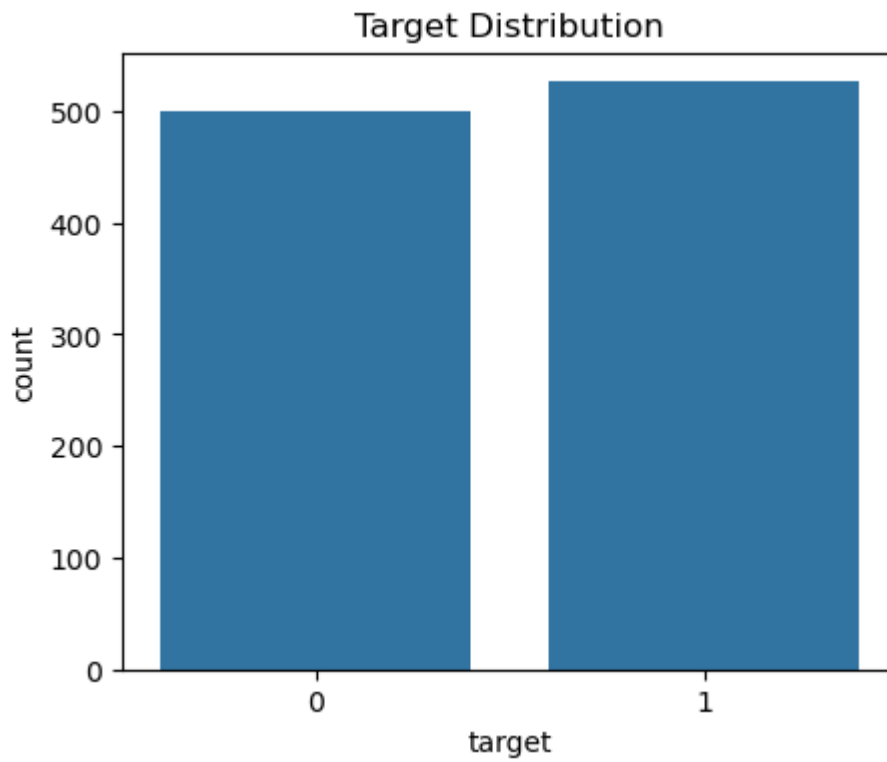
```
plt.tight_layout()
```

```
plt.show()
```

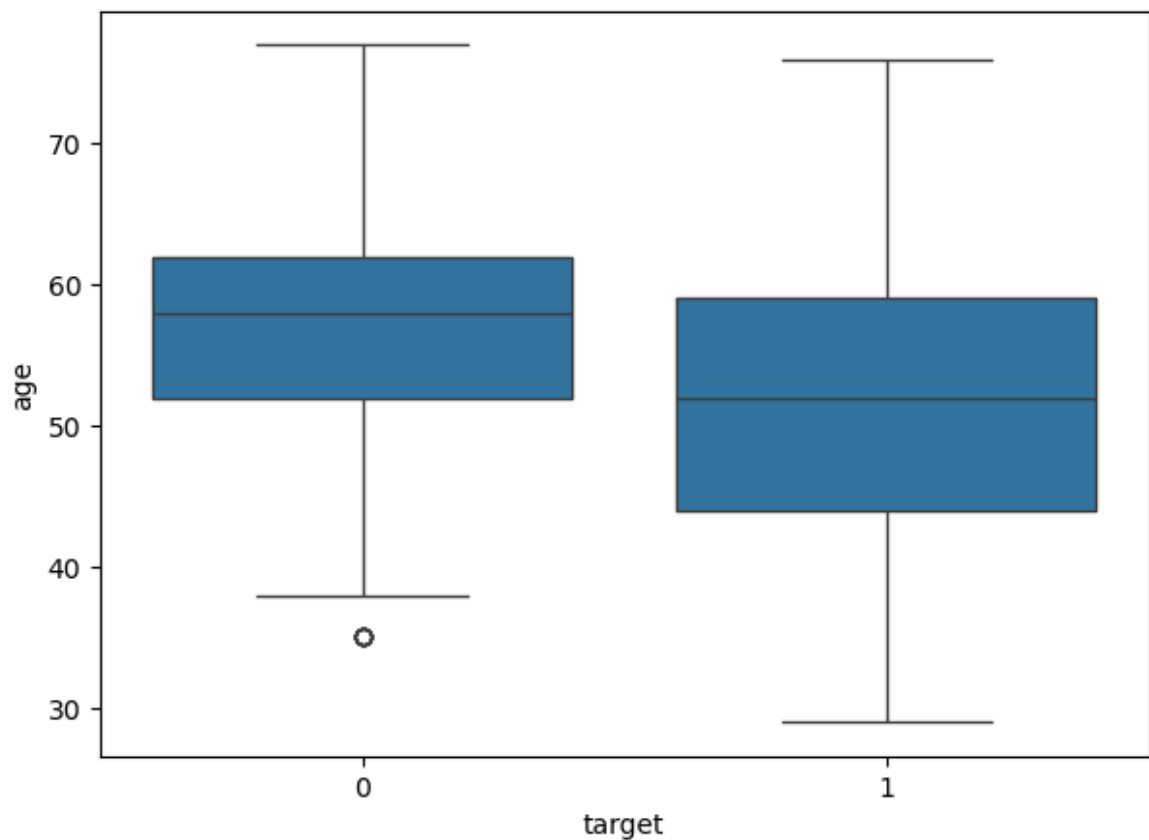


In [36]: *#13. Perform univariate analysis for categorical attributes.*

```
plt.figure(figsize=(5,4))
sns.countplot(x="target", data=df)
plt.title("Target Distribution")
plt.show()
```

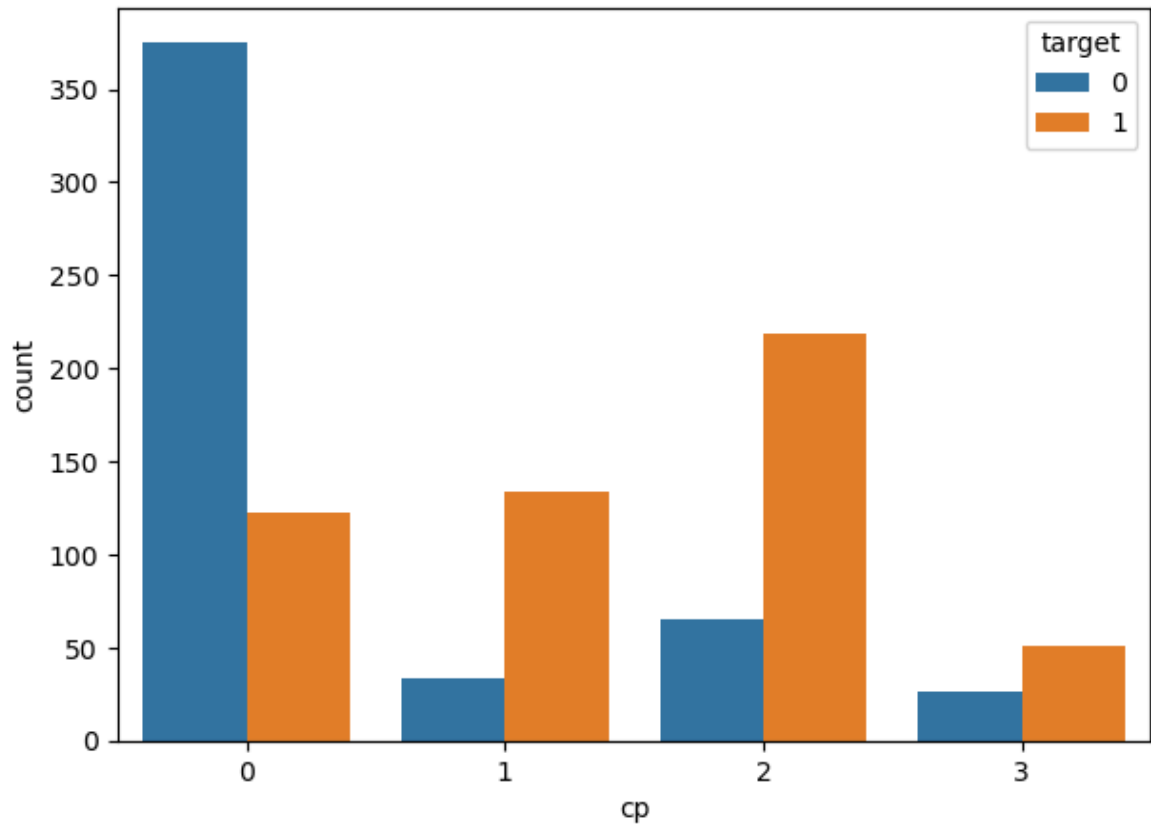


```
In [37]: #14. Perform bivariate analysis between each feature and the target variable.  
plt.figure(figsize=(7,5))  
sns.boxplot(x="target",  
            y="age",  
            data=df)  
plt.show()
```



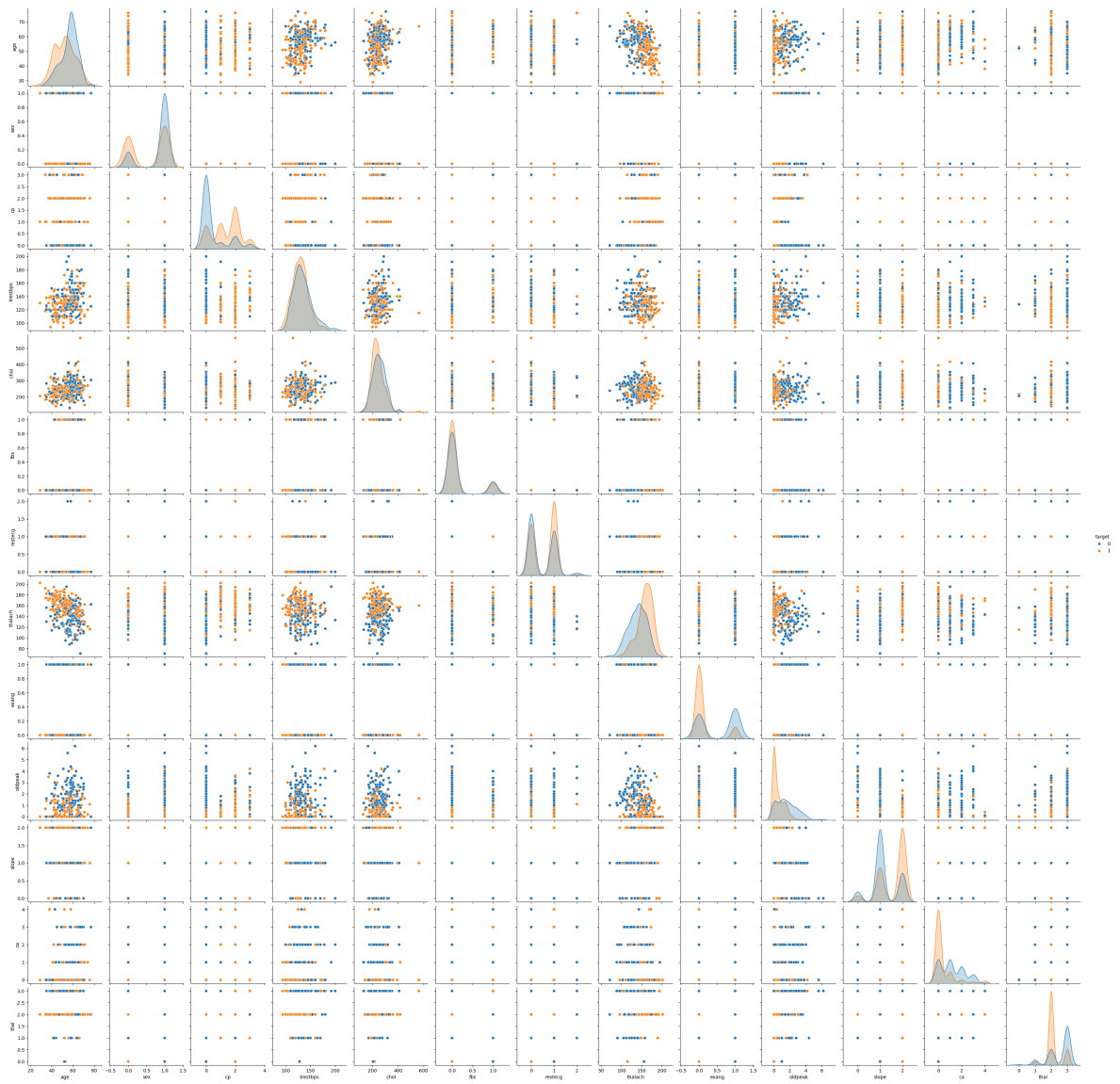
```
In [28]: #chest pain vs target  
plt.figure(figsize=(7,5))
```

```
sns.countplot(x="cp",hue="target",data=df)  
plt.show()
```



In [38]: *#15. Perform mul variate analysis using appropriate visualiza on techniques.*

```
sns.pairplot(df,hue="target")  
plt.show()
```



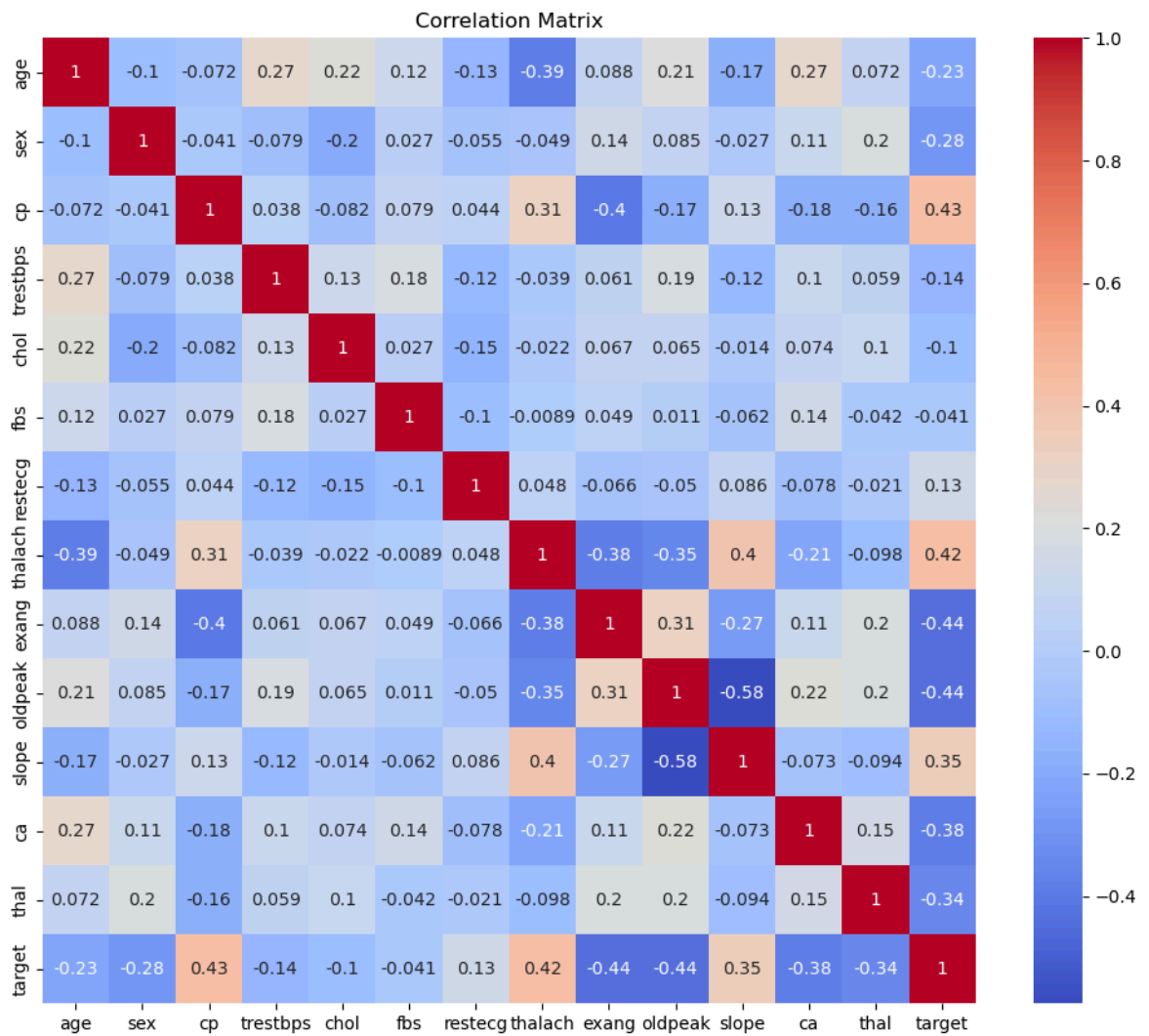
In [39]: #16. Generate a correlation on matrix for the numerical a ributes.
 corr = df.corr()
 corr

Out[39]:

	age	sex	cp	trestbps	chol	fbs	restecg	t
age	1.000000	-0.103240	-0.071966	0.271121	0.219823	0.121243	-0.132696	-0.0
sex	-0.103240	1.000000	-0.041119	-0.078974	-0.198258	0.027200	-0.055117	-0.0
cp	-0.071966	-0.041119	1.000000	0.038177	-0.081641	0.079294	0.043581	0.0
trestbps	0.271121	-0.078974	0.038177	1.000000	0.127977	0.181767	-0.123794	-0.0
chol	0.219823	-0.198258	-0.081641	0.127977	1.000000	0.026917	-0.147410	-0.0
fbs	0.121243	0.027200	0.079294	0.181767	0.026917	1.000000	-0.104051	-0.0
restecg	-0.132696	-0.055117	0.043581	-0.123794	-0.147410	-0.104051	1.000000	0.0
thalach	-0.390227	-0.049365	0.306839	-0.039264	-0.021772	-0.008866	0.048411	1.0
exang	0.088163	0.139157	-0.401513	0.061197	0.067382	0.049261	-0.065606	-0.0
oldpeak	0.208137	0.084687	-0.174733	0.187434	0.064880	0.010859	-0.050114	-0.0
slope	-0.169105	-0.026666	0.131633	-0.120445	-0.014248	-0.061902	0.086086	0.0
ca	0.271551	0.111729	-0.176206	0.104554	0.074259	0.137156	-0.078072	-0.0
thal	0.072297	0.198424	-0.163341	0.059276	0.100244	-0.042177	-0.020504	-0.0
target	-0.229324	-0.279501	0.434854	-0.138772	-0.099966	-0.041164	0.134468	0.0

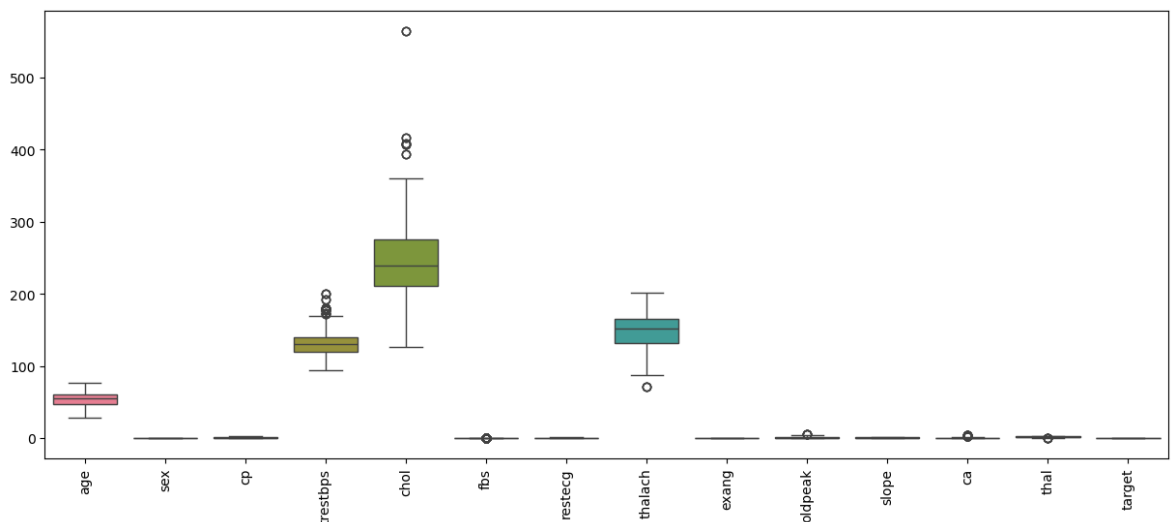


```
In [40]: #17. Visualize and interpret the correla on matrix using a heatmap.
plt.figure(figsize=(12,10))
sns.heatmap(corr,
             annot=True,
             cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



In [41]: #18. Detect outliers using Box Plot and IQR methods.

```
plt.figure(figsize=(15,6))
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.show()
```

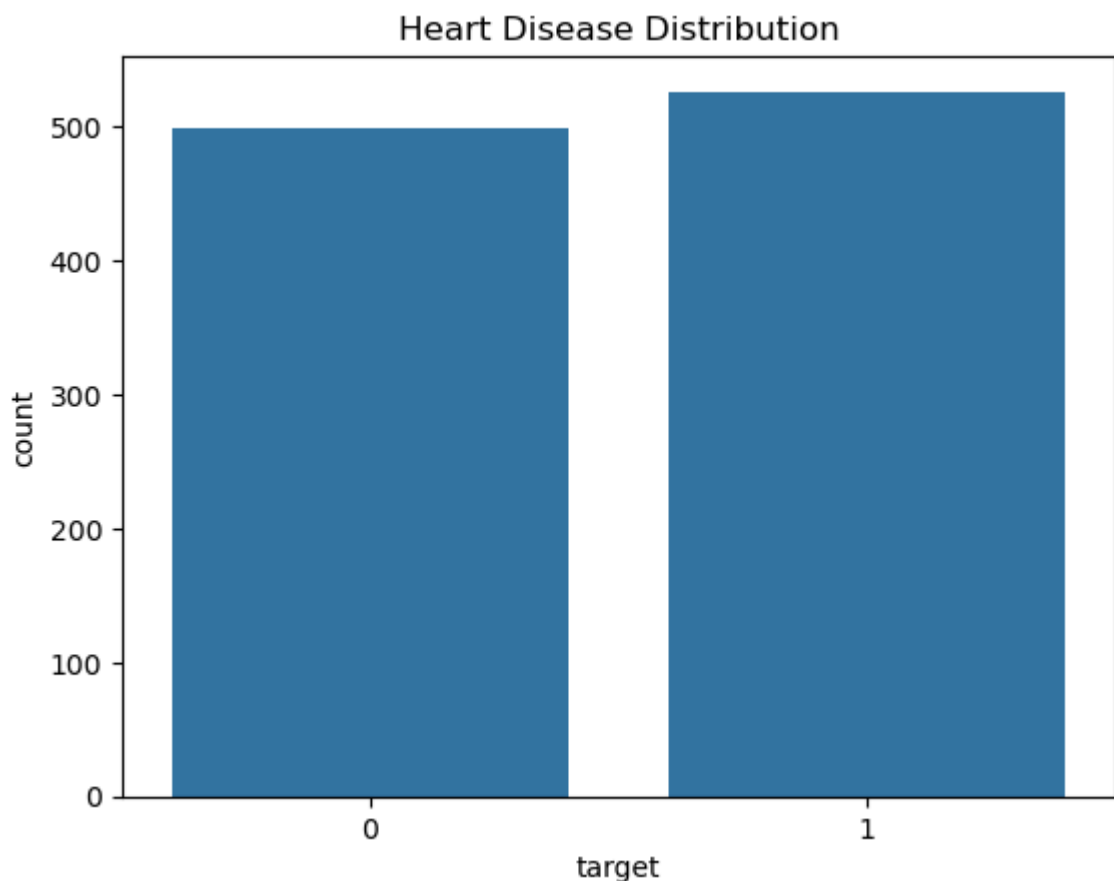


```
In [42]: Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
outliers = ((df < (Q1 - 1.5 * IQR)) |
```

```
(df > (Q3 + 1.5 * IQR)))  
outliers.sum()
```

```
Out[42]: age      0  
sex      0  
cp       0  
trestbps 30  
chol     16  
fbs      153  
restecg  0  
thalach  4  
exang    0  
oldpeak  7  
slope    0  
ca       87  
thal     7  
target   0  
dtype: int64
```

```
In [43]: #19. Analyze the distribu on of the target variable.  
sns.countplot(x="target",data=df)  
plt.title("Heart Disease Distribution")  
plt.show()
```



```
In [44]: #20. Summarize the observa ons obtained from EDA.  
df["target"].value_counts(normalize=True) * 100
```

```
Out[44]: target  
1      51.317073  
0      48.682927  
Name: proportion, dtype: float64
```

In [45]: *#Data Cleaning and Data Preprocessing Aim -To prepare the dataset for machine Le*

In [46]: *#21. Remove duplicate records from the dataset.*
 df = df.drop_duplicates()
 df.shape

Out[46]: (302, 14)

In [49]: *#22. Handle missing values using appropriate imputa on techniques.*
 df.fillna(df.mean(numeric_only=True),inplace=True)
 df.isnull().sum()

Out[49]: age 0
 sex 0
 cp 0
 trestbps 0
 chol 0
 fbs 0
 restecg 0
 thalach 0
 exang 0
 oldpeak 0
 slope 0
 ca 0
 thal 0
 target 0
 dtype: int64

In [50]: *#23. Convert categorical variables into numerical form using suitable encoding m*
 df = pd.get_dummies(df,drop_first=True)

In [51]: *#24. Apply feature standardiza on using StandardScaler.*
 X = df.drop("target",axis=1)
 y = df["target"]
 scaler = StandardScaler()
 X_standard = scaler.fit_transform(X)
 pd.DataFrame(X_standard).head()

Out[51]:

	0	1	2	3	4	5	6	7
0	-0.267966	0.682656	-0.935208	-0.376556	-0.667728	-0.418446	0.901657	0.806035
1	-0.157260	0.682656	-0.935208	0.478910	-0.841918	2.389793	-1.002541	0.237495
2	1.724733	0.682656	-0.935208	0.764066	-1.403197	-0.418446	0.901657	-1.074521
3	0.728383	0.682656	-0.935208	0.935159	-0.841918	-0.418446	0.901657	0.499898
4	0.839089	-1.464866	-0.935208	0.364848	0.919336	2.389793	0.901657	-1.905464

In [52]: *#25. Apply feature normaliza on using MinMaxScaler.*
 minmax = MinMaxScaler()
 X_normal = minmax.fit_transform(X)
 pd.DataFrame(X_normal).head()

Out[52]:

	0	1	2	3	4	5	6	7	8	9	10	11	
0	0.479167	1.0	0.0	0.292453	0.196347	0.0	0.5	0.740458	0.0	0.161290	1.0	0.50	1.
1	0.500000	1.0	0.0	0.433962	0.175799	1.0	0.0	0.641221	1.0	0.500000	0.0	0.00	1.
2	0.854167	1.0	0.0	0.481132	0.109589	0.0	0.5	0.412214	1.0	0.419355	0.0	0.00	1.
3	0.666667	1.0	0.0	0.509434	0.175799	0.0	0.5	0.687023	0.0	0.000000	1.0	0.25	1.
4	0.687500	0.0	0.0	0.415094	0.383562	1.0	0.5	0.267176	0.0	0.306452	0.5	0.75	0.

In [53]: *#26. Compare the effect of different feature scaling techniques on the dataset.*
 print("Standardized Data")
 pd.DataFrame(X_standard).head()

Standardized Data

Out[53]:

	0	1	2	3	4	5	6	7
0	-0.267966	0.682656	-0.935208	-0.376556	-0.667728	-0.418446	0.901657	0.806035
1	-0.157260	0.682656	-0.935208	0.478910	-0.841918	2.389793	-1.002541	0.237495
2	1.724733	0.682656	-0.935208	0.764066	-1.403197	-0.418446	0.901657	-1.074521
3	0.728383	0.682656	-0.935208	0.935159	-0.841918	-0.418446	0.901657	0.499898
4	0.839089	-1.464866	-0.935208	0.364848	0.919336	2.389793	0.901657	-1.905464

In [54]: print("Normalized Data")
 pd.DataFrame(X_normal).head()

Normalized Data

Out[54]:

	0	1	2	3	4	5	6	7	8	9	10	11	
0	0.479167	1.0	0.0	0.292453	0.196347	0.0	0.5	0.740458	0.0	0.161290	1.0	0.50	1.
1	0.500000	1.0	0.0	0.433962	0.175799	1.0	0.0	0.641221	1.0	0.500000	0.0	0.00	1.
2	0.854167	1.0	0.0	0.481132	0.109589	0.0	0.5	0.412214	1.0	0.419355	0.0	0.00	1.
3	0.666667	1.0	0.0	0.509434	0.175799	0.0	0.5	0.687023	0.0	0.000000	1.0	0.25	1.
4	0.687500	0.0	0.0	0.415094	0.383562	1.0	0.5	0.267176	0.0	0.306452	0.5	0.75	0.

In [56]: *#27. Remove irrelevant or redundant attributes.*
 if "id" in df.columns:
 df.drop(columns=["id"], inplace=True)
 print("ID column removed.")
 else:
 print("No ID column found. No attributes removed.")

No ID column found. No attributes removed.

In [57]: *#28. Validate the processed dataset using Great Expectations by verifying schema*
 !pip install great_expectations

Collecting great_expectations

Downloading great_expectations-1.19.1-py3-none-any.whl.metadata (9.7 kB)
Requirement already satisfied: altair<7.0.0,>=5.0.0 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (5.0.1)
Requirement already satisfied: cryptography>=3.2 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (43.0.0)
Requirement already satisfied: jinja2>=3 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (3.1.4)
Requirement already satisfied: jsonschema>=2.5.1 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (4.23.0)
Collecting marshmallow<4.0.0,>=3.7.1 (from great_expectations)
Downloading marshmallow-3.26.2-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: mistune>=0.8.4 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (2.0.4)
Requirement already satisfied: numpy>=1.22.4 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (1.26.4)
Requirement already satisfied: packaging in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (24.1)
Requirement already satisfied: pandas>=1.3.0 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (2.2.2)
Requirement already satisfied: pydantic>=1.10.7 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (2.8.2)
Requirement already satisfied: pyparsing!=3.2.4,>=2.4 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (3.1.2)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (2.9.0.post0)
Requirement already satisfied: requests>=2.20 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (2.32.3)
Requirement already satisfied: ruamel.yaml>=0.16 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (0.18.6)
Requirement already satisfied: scipy>=1.6.0 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (1.13.1)
Requirement already satisfied: tqdm>=4.59.0 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (4.66.5)
Requirement already satisfied: typing-extensions>=4.1.0 in c:\users\yaswa\anaconda3\lib\site-packages (from great_expectations) (4.15.0)
Collecting tzlocal>=1.2 (from great_expectations)
Downloading tzlocal-5.4.4-py3-none-any.whl.metadata (7.7 kB)
Requirement already satisfied: toolz in c:\users\yaswa\anaconda3\lib\site-packages (from altair<7.0.0,>=5.0.0->great_expectations) (0.12.0)
Requirement already satisfied: cffi>=1.12 in c:\users\yaswa\anaconda3\lib\site-packages (from cryptography>=3.2->great_expectations) (1.17.1)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\yaswa\anaconda3\lib\site-packages (from jinja2>=3->great_expectations) (2.1.3)
Requirement already satisfied: attrs>=22.2.0 in c:\users\yaswa\anaconda3\lib\site-packages (from jsonschema>=2.5.1->great_expectations) (23.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\users\yaswa\anaconda3\lib\site-packages (from jsonschema>=2.5.1->great_expectations) (2023.7.1)
Requirement already satisfied: referencing>=0.28.4 in c:\users\yaswa\anaconda3\lib\site-packages (from jsonschema>=2.5.1->great_expectations) (0.30.2)
Requirement already satisfied: rpds-py>=0.7.1 in c:\users\yaswa\anaconda3\lib\site-packages (from jsonschema>=2.5.1->great_expectations) (0.10.6)
Requirement already satisfied: pytz>=2020.1 in c:\users\yaswa\anaconda3\lib\site-packages (from pandas>=1.3.0->great_expectations) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\yaswa\anaconda3\lib\site-packages (from pandas>=1.3.0->great_expectations) (2023.3)
Requirement already satisfied: annotated-types>=0.4.0 in c:\users\yaswa\anaconda3\lib\site-packages (from pydantic>=1.10.7->great_expectations) (0.6.0)
Requirement already satisfied: pydantic-core==2.20.1 in c:\users\yaswa\anaconda3

```

\lib\site-packages (from pydantic>=1.10.7->great_expectations) (2.20.1)
Requirement already satisfied: six>=1.5 in c:\users\yaswa\anaconda3\lib\site-pack
ages (from python-dateutil>=2.8.1->great_expectations) (1.16.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\yaswa\anacond
a3\lib\site-packages (from requests>=2.20->great_expectations) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\yaswa\anaconda3\lib\site-
packages (from requests>=2.20->great_expectations) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\yaswa\anaconda3\lib
\site-packages (from requests>=2.20->great_expectations) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\yaswa\anaconda3\lib
\site-packages (from requests>=2.20->great_expectations) (2024.8.30)
Requirement already satisfied: ruamel.yaml.clib>=0.2.7 in c:\users\yaswa\anaconda
3\lib\site-packages (from ruamel.yaml>=0.16->great_expectations) (0.2.8)
Requirement already satisfied: colorama in c:\users\yaswa\anaconda3\lib\site-pack
ages (from tqdm>=4.59.0->great_expectations) (0.4.6)
Requirement already satisfied: pycparser in c:\users\yaswa\anaconda3\lib\site-pac
kages (from cffi>=1.12->cryptography>=3.2->great_expectations) (2.21)
Downloading great_expectations-1.19.1-py3-none-any.whl (4.9 MB)
----- 0.0/4.9 MB ? eta -:--:--
-- ----- 0.3/4.9 MB ? eta -:--:--
----- 0.5/4.9 MB 2.4 MB/s eta 0:00:02
----- 2.4/4.9 MB 4.5 MB/s eta 0:00:01
----- 4.7/4.9 MB 6.6 MB/s eta 0:00:01
----- 4.9/4.9 MB 6.5 MB/s eta 0:00:00
Downloading marshmallow-3.26.2-py3-none-any.whl (50 kB)
Downloading tzlocal-5.4.4-py3-none-any.whl (18 kB)
Installing collected packages: tzlocal, marshmallow, great_expectations
Successfully installed great_expectations-1.19.1 marshmallow-3.26.2 tzlocal-5.4.4

```

```

In [4]: #28. Validate the processed dataset using Great Expecta ons by verifying schema
#missing values
import great_expectations as gx
gx_df = gx.from_pandas(df)
gx_df.expect_column_values_to_not_be_null("age")

```

```

Out[4]: {
  "success": true,
  "result": {
    "element_count": 1025,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "partial_unexpected_list": []
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}

```

```

In [5]: #datatypes
gx_df.expect_column_values_to_be_of_type("age", "int")

```

```
Out[5]: {
  "success": false,
  "result": {
    "observed_value": "int64"
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
In [6]: # 4. Value ranges
```

```
gx_df.expect_column_values_to_be_between("age", min_value=20, max_value=100)
gx_df.expect_column_values_to_be_between("chol", min_value=100, max_value=600)
```

```
Out[6]: {
  "success": true,
  "result": {
    "element_count": 1025,
    "missing_count": 0,
    "missing_percent": 0.0,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "unexpected_percent_nonmissing": 0.0,
    "partial_unexpected_list": []
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
In [7]: # 5. Categorical constraints
```

```
gx_df.expect_column_values_to_be_in_set("target", [0, 1])
```

```
# Display validation results
```

```
print(gx_df.validate())
```



```

{
  "success": false,
  "results": [
    {
      "success": true,
      "expectation_config": {
        "expectation_type": "expect_column_values_to_not_be_null",
        "kwargs": {
          "column": "age",
          "result_format": "BASIC"
        },
      },
      "meta": {}
    },
    {
      "result": {
        "element_count": 1025,
        "unexpected_count": 0,
        "unexpected_percent": 0.0,
        "unexpected_percent_total": 0.0,
        "partial_unexpected_list": []
      },
      "meta": {},
      "exception_info": {
        "raised_exception": false,
        "exception_message": null,
        "exception_traceback": null
      }
    },
    {
      "success": false,
      "expectation_config": {
        "expectation_type": "expect_column_values_to_be_of_type",
        "kwargs": {
          "column": "age",
          "type_": "int",
          "result_format": "BASIC"
        },
      },
      "meta": {}
    },
    {
      "result": {
        "observed_value": "int64"
      },
      "meta": {},
      "exception_info": {
        "raised_exception": false,
        "exception_message": null,
        "exception_traceback": null
      }
    },
    {
      "success": true,
      "expectation_config": {
        "expectation_type": "expect_column_values_to_be_between",
        "kwargs": {
          "column": "age",
          "min_value": 20,
          "max_value": 100,
          "result_format": "BASIC"
        },
      },
      "meta": {}
    },
  ],
}

```

```

"result": {
  "element_count": 1025,
  "missing_count": 0,
  "missing_percent": 0.0,
  "unexpected_count": 0,
  "unexpected_percent": 0.0,
  "unexpected_percent_total": 0.0,
  "unexpected_percent_nonmissing": 0.0,
  "partial_unexpected_list": []
},
"meta": {},
"exception_info": {
  "raised_exception": false,
  "exception_message": null,
  "exception_traceback": null
}
},
{
  "success": true,
  "expectation_config": {
    "expectation_type": "expect_column_values_to_be_between",
    "kwargs": {
      "column": "chol",
      "min_value": 100,
      "max_value": 600,
      "result_format": "BASIC"
    },
    "meta": {}
  },
  "result": {
    "element_count": 1025,
    "missing_count": 0,
    "missing_percent": 0.0,
    "unexpected_count": 0,
    "unexpected_percent": 0.0,
    "unexpected_percent_total": 0.0,
    "unexpected_percent_nonmissing": 0.0,
    "partial_unexpected_list": []
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_message": null,
    "exception_traceback": null
  }
},
{
  "success": true,
  "expectation_config": {
    "expectation_type": "expect_column_values_to_be_in_set",
    "kwargs": {
      "column": "target",
      "value_set": [
        0,
        1
      ],
      "result_format": "BASIC"
    },
    "meta": {}
  },

```

```

    "result": {
      "element_count": 1025,
      "missing_count": 0,
      "missing_percent": 0.0,
      "unexpected_count": 0,
      "unexpected_percent": 0.0,
      "unexpected_percent_total": 0.0,
      "unexpected_percent_nonmissing": 0.0,
      "partial_unexpected_list": []
    },
    "meta": {},
    "exception_info": {
      "raised_exception": false,
      "exception_message": null,
      "exception_traceback": null
    }
  }
],
"evaluation_parameters": {},
"statistics": {
  "evaluated_expectations": 5,
  "successful_expectations": 4,
  "unsuccessful_expectations": 1,
  "success_percent": 80.0
},
"meta": {
  "great_expectations_version": "0.18.21",
  "expectation_suite_name": "default",
  "run_id": {
    "run_name": null,
    "run_time": "2026-07-26T14:29:33.761995+05:30"
  },
  "batch_kwargs": {
    "ge_batch_id": "d6aa4c97-88cf-11f1-a89f-3003c883d6d2"
  },
  "batch_markers": {},
  "batch_parameters": {},
  "validation_time": "20260726T085933.761995Z",
  "expectation_suite_meta": {
    "great_expectations_version": "0.18.21"
  }
}
}
}

```

```

In [14]: expected_columns = [
    "age", "sex", "cp", "trestbps", "chol", "fbs",
    "restecg", "thalach", "exang", "oldpeak",
    "slope", "ca", "thal", "target"
]

gx_df.expect_table_columns_to_match_ordered_list(expected_columns)

```

```
Out[14]: {
  "success": true,
  "result": {
    "observed_value": [
      "age",
      "sex",
      "cp",
      "trestbps",
      "chol",
      "fbs",
      "restecg",
      "thalach",
      "exang",
      "oldpeak",
      "slope",
      "ca",
      "thal",
      "target"
    ]
  },
  "meta": {},
  "exception_info": {
    "raised_exception": false,
    "exception_traceback": null,
    "exception_message": null
  }
}
```

```
In [38]: #29. Save the cleaned dataset for subsequent experiments.
df.to_csv("heart_cleaned.csv", index=False)

print("Dataset Saved Successfully-231FA04B07")
```

Dataset Saved Successfully-231FA04B07

```
In [ ]:
```